

JavaScript

MasterSEO
by BIGSEO

TU CAM

HTML

ÍNDICE

JS

Entorno cliente-servidor

Ejecutando JS

Sintaxis JS

Variables

Operadores

Estructuras de control

Mostrar mensajes

Pedir datos al usuario

Eventos

MasterSEO
by BIGSEO

TU CAM

JavaScript

JavaScript

¿Qué es JS?

- Lenguaje de programación utilizado para implementar funciones en páginas web.
- Permite crear contenido de actualización dinámica, animaciones, cálculos, etc.
- Es un lenguaje que se ejecuta en entorno cliente.

Entendemos como “cliente” a cualquier programa que se conecte a internet para obtener cualquier tipo de información.



TU CAM

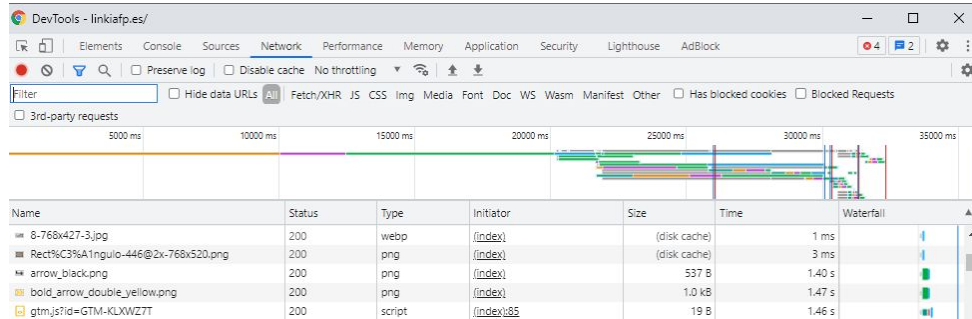
ENTORNO CLIENTE - SERVIDOR

Entorno cliente - Servidor

De la misma forma se define cliente como cualquier programa que realiza una petición por internet, se define servidor como entidad responsable de servir un recurso pedido.

Un ejemplo común sería el siguiente:

- El navegador funciona como cliente cuando el usuario visita una página web.
- El servidor web responde a todas las peticiones (imágenes, textos, estilos, etc.) para que el cliente muestre en pantalla lo que el usuario está buscando.



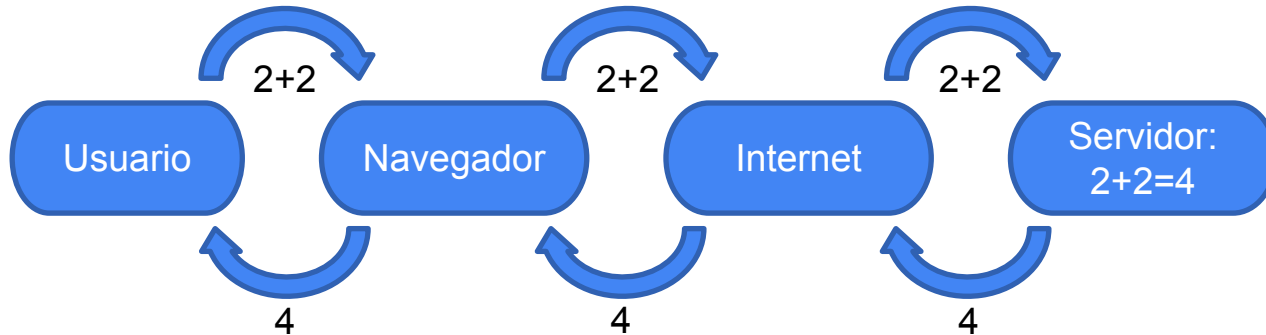
TU CAM

Entorno cliente - Servidor

En los inicios de internet, los navegadores únicamente se limitaban a mostrar una interfaz para que los usuarios pudiesen consultar los contenidos alojados en diferentes servidores.

Por lo tanto, los navegadores (clientes) no ejecutaban código de programación.

Si analizamos un ejemplo simple, como el desarrollo de una calculadora sin lenguaje cliente, se obtiene el siguiente esquema de conexión:



TU CAM

Entorno cliente - Servidor

Es necesario tener en cuenta que la realización de peticiones a servidores son operaciones lentas y costosas y por eso nace la necesidad de que el navegador pueda realizar ciertas operaciones.

Actualmente, gracias a los lenguajes de programación en entorno cliente, se pueden realizar muchos cálculos sin necesidad de establecer conexiones por internet y esto, agiliza mucho la navegabilidad del usuario por las páginas.

- Ejemplo de clientes:
 - Navegador web
 - Cliente FTP
 - Cliente de Chat
 - Cliente de correo



TU CAM

Entorno cliente - Servidor

- Ejemplo de servidores:
 - Servidor web
 - Servidor FCT
 - Servidor de Chat
 - Servidor de correo



TU CAM

Entorno cliente - Servidor

A nivel web, los principales protocolos que se ejecutan en entorno servidor son los siguientes:

- PHP: lenguaje de programación Open Source. Suele ser utilizado para generar códigos HTML que el navegador interpretará como una página web. Los archivos se identifican con la extensión “.php”.
- JSP (Java Server Page): tecnología que genera páginas web ejecutando código Java. Muy potente y robusta. Los archivos se identifican con la extensión “.jsp”.
- ASP.net: framework de Microsoft. Robustez y seguridad respaldada por esta empresa. Alto coste de servidores y licencias de desarrollo. Los archivos se identifican con la extensión “.asp”.

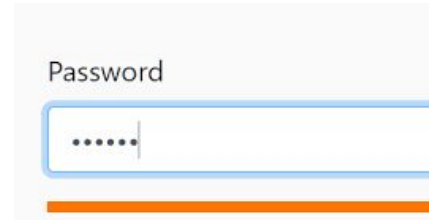


TU CAM

Ejecutando JavaScript

Ejecutando JavaScript

- El objetivo de JavaScript consiste en facilitar la interacción web con el usuario, es decir, mejorar su usabilidad y accesibilidad. Por ello, todos los códigos JavaScript siempre van a ir vinculados a un documento HTML.
- Un documento HTML puede tener vinculados varios JS y, a la vez, un JS puede estar enlazado desde varios HTML.
- Al ser un lenguaje cliente, el usuario siempre tendrá acceso a ver los códigos JS que se ejecuten.
- Por este motivo, es importante no utilizar JS para mejorar la seguridad de una página web: validaciones de campos, control de contraseñas, etc.



Password

TU CAM

Ejecutando JavaScript

Para integrar un código JavaScript dentro de un documento HTML tenemos 3 maneras:

1. Abriendo una etiqueta “<script>” dentro del código HTML y añadiendo el código dentro.

```
<!DOCTYPE html>
<head>
  <title>Document</title>
  <script>
    // código JavaScript
  </script>
</head>
<body>
```

TU CAM

Ejecutando JavaScript

Para integrar un código JavaScript dentro de un documento HTML tenemos 3 maneras:

1. Abriendo una etiqueta “<script>” dentro del código HTML y añadiendo el código dentro.

```
<!DOCTYPE html>
<head>
  <title>Document</title>
  <script>
    // código JavaScript
  </script>
</head>
<body>
```

Esta forma es muy sencilla pero no podemos reutilizar este código para otra página web.

TU CAM

Ejecutando JavaScript

2. La segunda forma es tener el código JavaScript en un documento “.js” y enlazándolo de la siguiente forma:

```
<!DOCTYPE html>
<head>
  <title>Documento HTML</title>
  <script src="ejemplo.js"></script>
</head>
<body>

</body>
```

De esta forma se podrá enlazar este script desde todas las páginas que necesitemos. Es la manera más utilizada y recomendable.

TU CAM

Ejecutando JavaScript

3. La tercera manera consiste en vincular dentro del mismo código HTML un código JavaScript a una cierta acción realizada sobre un elemento HTML.

A esta acción se le llama “eventos”.

Para crear éste vínculo en el mismo HTML:

- Se ha de añadir al elemento sobre el que vamos a realizar la acción un atributo que indique qué acción realizaremos sobre él.
- A este atributo le daremos como valor el código JavaScript que queremos que se ejecute al realizarse la acción.

TU CAM

Ejecutando JavaScript

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <h1>Evento Onclick</h1>
    <button onclick="document.getElementById('demo').innerHTML = 'Hola a todos';">
      Click me</button>
    <p id="demo"></p>
  </body>
</html>
```

Evento Onclick

Click me

Evento Onclick

Click me

Hola a todos

TU CAM

Ejecutando JavaScript

```
<!DOCTYPE html>
<html>
  <head>
    <script>
      function funcion() {
        document.getElementById("demo").innerHTML = "Hola a todos";
      }
    </script>
  </head>
  <body>
    <h1>Evento Onclick</h1>
    <button onclick="funcion()">Click me</button>
    <p id="demo"></p>
  </body>
</html>
```

Evento Onclick

Click me

Evento Onclick

Click me

Hola a todos

TU CAM

SINTAXIS

Sintaxis

- Es un lenguaje case-sensitive: Si se escribe alguna variable en mayúscula, se ha de respetar en todo el código.
- Las sentencias se pueden separar con saltos de línea o con el signo “;”. Es recomendable utilizar la segunda opción para poder comprimir el código posteriormente eliminando los saltos de línea.
- El nombre de las variables ha de iniciar por letra o “_”. Se recomienda reservar los inicios con “_” para los constructores*.
- Las variables compuestas se suelen escribir utilizando la convención “lowe camel case”. Consiste en concatenar todas las palabras usando mayúsculas en las primeras letras de las mismas.

Ej: “mi nombre” □ “miNombre”

El método constructor es un método especial para crear e inicializar un objeto creado a partir de una clase.

TU CAM

Sintaxis

Existen una serie de palabras reservadas que utiliza el lenguaje internamente y no se pueden utilizar como variables, etiquetas o nombres de funciones:

abstract	arguments	boolean	break	byte
case	catch	char	class*	const
continue	debugger	default	delete	do
double	else	enum*	eval	export*
extends*	false	final	finally	float
for	function	goto	if	implements
import*	in	instanceof	int	interface
let	long	native	new	null
package	private	protected	public	return
short	static	super*	switch	synchronized
this	throw	throws	transient	true
try	typeof	var	void	volatile
while	with	yield		

https://www.w3bai.com/es/js/js_reserved.html

TU CAM

Sintaxis

Comentarios: textos que podemos escribir en el código y que no se ejecutan.

- Comentar una línea con “//”

```
// Change heading:
document.getElementById("myH").innerHTML = "My First Page";
// Change paragraph:
document.getElementById("myP").innerHTML = "My first paragraph.";

var x = 5;      // Declare x, give it the value of 5
var y = x + 2;  // Declare y, give it the value of x + 2
```

- Comentar múltiples líneas con “/*” y “*/”

```
/*
The code below will change
the heading with id = "myH"
and the paragraph with id = "myP"
in my web page:
*/
document.getElementById("myH").innerHTML = "My First Page";
document.getElementById("myP").innerHTML = "My first paragraph.";
```

Sintaxis

Declaración de variables: las variables se declaran con las palabras “let” o “var” seguido del nombre que le queramos dar a la variable.

JavaScript es un lenguaje de programación “no tipado”, por lo tanto, no es necesario asignar el tipo de variable a utilizar según el dato que se vaya a almacenar. La misma variable puede almacenar varios tipos de datos durante la ejecución del programa.

```
var nombre = "Ivan";  
let edad = 25;
```

La declaración de variables con “var” y “let” son diferentes:

- Las variables declaradas con “let” se limitan al bloque de código que se ha definido. Las variables declaradas con “var” son globales.
- Una variable declarada con “let” únicamente se puede declarar una vez.

TU CAM

VARIABLES

Variables

Aunque no es necesario especificar el tipo de dato que almacenará una variable, JavaScript ofrece una serie de tipos de variables predefinidos:

- **Number:** permite almacenar números enteros o decimales separados por puntos. Si escribimos el número entre comillas se interpretará como un String. Es el único tipo de variable numérica que dispone JavaScript
- **String:** Permite almacenar un texto. El texto se ha de escribir entre comillas simples o dobles. Se pueden concatenar con el símbolo “+”.

Cualquier tipo de variable concatenada con una String se convierte en String.

TU CAM

Variables

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>

<p id="resultado1"></p>
<p id="resultado2"></p>
<p id="resultado3"></p>

<script>
var ejemplo1 = 5;
var ejemplo2 = 6;
var ejemplo3 = "5";
var ejemplo4 = "6";
var resultado1 = ejemplo1 + ejemplo2;
var resultado2 = ejemplo3 + ejemplo4;
var resultado3 = ejemplo2 + ejemplo4;

document.getElementById("resultado1").innerHTML = "Resultado 1: " + resultado1;
document.getElementById("resultado2").innerHTML = "Resultado 2: " + resultado2;
document.getElementById("resultado3").innerHTML = "Resultado 3: " + resultado3;

</script>

</body>
</html>
```

JavaScript Variables

Resultado 1: 11

Resultado 2: 56

Resultado 3: 66

TU CAM

Variables

- Boolean: permite almacenar dos valores: “true” o “false”. En este tipo de variable no se ha de asignar el valor entre comillas.

```
let activado = true;
```

- Array: Permite almacenar múltiples valores en una misma variable.

```
<script>  
  var cars = ["Saab", "Volvo", "BMW"];  
</script>
```

```
var cars = [  
  "Saab",  
  "Volvo",  
  "BMW"  
];
```

```
var cars = [];  
cars[0] = "Saab";  
cars[1] = "Volvo";  
cars[2] = "BMW";
```

TU CAM

Variables

Es posible convertir variables de un tipo a otro.

```
var texto="" + 150.26;           //texto vale "150.26"  
var entero= parseInt(texto);    //entero vale 150  
var decimal = parseFloat(texto); //decimal vale 150.26
```

Para convertir a String es suficiente con concatenar la variable con una cadena vacía.
Para convertir el resto de tipos se utilizan funciones específicas que incorpora el lenguaje.

<u>Number()</u>	Converts an object's value to a number
<u>parseFloat()</u>	Parses a string and returns a floating point number
<u>parseInt()</u>	Parses a string and returns an integer
<u>String()</u>	Converts an object's value to a string

https://www.w3schools.com/jsref/jsref_obj_global.asp

TU CAM

OPERADORES

Operadores

Los operadores aritméticos son comunes en la mayoría de lenguajes y nos permiten operar con las variables.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement

```
let a = 3;  
let x = (100 + 50) * a;
```

https://www.w3schools.com/js/js_arithmetic.asp

TU CAM

Operadores

Los operadores de asignación permiten asignar valores a variables. También son comunes en la mayoría de lenguajes.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

```
let x = 10;  
let y = 5;  
x *= y --> 50
```

TU CAM

Operadores

Los operadores de comparación permiten detectar las diferencias entre variables o valores.

Operator	Description	Comparing	Returns
==	equal to	x == 8	false
		x == 5	true
===	equal value and equal type	x === "5"	false
		x === 5	true
!=	not equal	x != 8	true
!==	not equal value or not equal type	x !== "5"	true
		x !== 5	false
>	greater than	x > 8	false
<	less than	x < 8	true
>=	greater than or equal to	x >= 8	false
<=	less than or equal to	x <= 8	true

```
<button onclick="myFunction()">Try it</button>
```

```
<p id="demo1"></p>
<p id="demo2"></p>
```

```
<script>
function myFunction() {
  var x = 5;
  document.getElementById("demo1").innerHTML = (x === "5"); //Resultado= false
  document.getElementById("demo2").innerHTML = (x == "5"); //Resultado= true
}
</script>
```

TU CAM

https://www.w3schools.com/jsref/jsref_operators.asp

Operadores

Los operadores lógicos permiten detectar comparaciones lógicas entre variables o valores.

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
	or	(x === 5 y === 5) is false
!	not	!(x === y) is true

Existen más tipos de operadores que se pueden consultar desde el enlace inferior. Los principales y más básicos son los que se han comentado hasta el momento.

https://www.w3schools.com/jsref/jsref_operators.asp

TU CAM

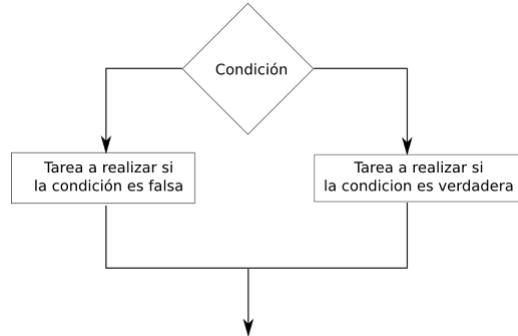
ESTRUCTURAS DE CONTROL

Estructuras de Control de flujo

Las estructuras de control de flujo permiten alterar el flujo de ejecución de un programa.

Es posible conseguir que un código se ejecute o no, se repita múltiples veces, deje de ejecutarse o se ejecute únicamente si se detecta algún error.

JavaScript incorpora las principales estructuras de control que se pueden encontrar también en la mayoría de lenguajes de programación.



TU CAM

Estructuras de Control de flujo

If / Else

Permite validar una expresión booleana y en base al resultado (true/false) seguir con la ejecución de un código u otro. En caso de que el resultado sea “false”, el código a ejecutar es el de dentro de la sentencia “else”.

- La sentencia “else” es opcional.
- Se pueden anidar varias sentencias “if/else”

```
if (hour < 18) {  
    greeting = "Good day";  
}
```

```
if (hour < 18) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

```
if (time < 10) {  
    greeting = "Good morning";  
} else if (time < 20) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```

TU CAM

Estructuras de Control de flujo

Switch / Case

Permite buscar una coincidencia entre una variable y distintos valores. A cada valor o “case”, le indicaremos el código a ejecutar en caso de coincidencia.

- Después de ejecutar un contenido, se ejecutará el siguiente “case” si no se encuentra la sentencia “break;”
- Se pueden añadir un case default que se ejecuta cuando no se encuentre ninguna coincidencia

```
let day;  
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";  
    break;  
  case 5:  
    day = "Friday";  
    break;  
  case 6:  
    day = "Saturday";  
    break;  
  default:  
    day = "error";  
}
```

TU CAM

Estructuras de Control de flujo

While

Permite definir un bloque de código a ejecutar mientras se cumpla una condición.

```
<p id="demo"></p>

<script>
let text = "";
let i = 0;
while (i < 10) {
  text += "<br>The number is " + i;
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>
```

JavaScript While Loop

The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9

TU CAM

Estructuras de Control de flujo

For

Permite definir un bloque de código a ejecutar un número determinado de veces.

```
<h2>JavaScript For Loop</h2>

<p id="demo"></p>

<script>
let text = "";

for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>
```

JavaScript For Loop

The number is 0
The number is 1
The number is 2
The number is 3
The number is 4

TU CAM

Estructuras de Control de flujo

Do While

Permite definir un bloque de código a ejecutar mientras se cumpla una condición.

A diferencia del while, el do while evalúa la condición después de haber ejecutado el bloque de código.

El bloque de código se ejecuta como mínimo 1 vez.

```
<script>
  var letra;

  do
  {
    letra = prompt("Introduce una letra");
  }
  while (!isNaN(letra));

</script>
```

Una página insertada en esta dice

Introduce una letra

Aceptar

Cancelar

```
<script>
console.log(isNaN("1")); //false
console.log(!isNaN("1")); //true
console.log(isNaN("a")); //true
console.log(!isNaN("a")); //false
</script>
```

TU CAM

Estructuras de Control de flujo

Try / catch / finally

Nos permite programar acciones a realizar si se produce algún error durante la ejecución de un código.

- Try: contiene código a controlar posibles errores
- Catch: acciones a realizar cuando se produzca el error
- Finally: parte opcional. Acciones que siempre queramos realizar, se haya producido o no el error

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript try catch</h2>

<p>Please input a number between 5 and 10:</p>

<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>

<p id="p01"></p>

<script>
function myFunction() {
  var message = document.getElementById("p01");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;
  try {
    if(x == "") throw "is empty";
    if(isNaN(x)) throw "is not a number";
    x = Number(x);
    if(x > 10) throw "is too high";
    if(x < 5) throw "is too low";
  }
  catch(err) {
    message.innerHTML = "Input " + err;
  }
  finally {
    document.getElementById("demo").value = "";
  }
}
</script>

</body>
</html>
```

JavaScript try catch

Please input a number between 5 and 10:

Input is not a number

TU CAM

MOSTRAR MENSAJES

Mostrar mensajes

La forma más fácil de mostrar mensajes en pantalla es lanzando una pantalla emergente con la siguiente sintaxis:

```
alert("Mensaje a mostrar al usuario!");
```

Una página insertada en esta dice

Mensaje a mostrar al usuario!

Aceptar

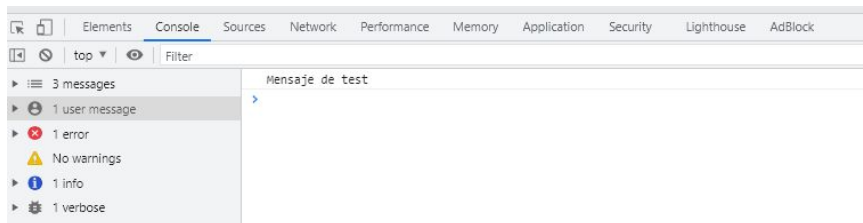
Esta forma de mostrar la información es la menos recomendable:

- Es muy molesta y poco usable.
- Detiene la ejecución del código JS y la carga de la web hasta que el usuario no afecta.

TU CAM

Mostrar mensajes

Si queremos mostrar mensajes de depuración, no visibles para a los usuarios en general, se pueden utilizar los mensajes de consola de la siguiente manera:



```
console.log("Mensaje de test");
```

Esta forma es la más recomendable a utilizar mientras se programa y se necesita mostrar información de forma rápida para conocer el estado del programa.

- Estos mensajes sólo se muestran en las herramientas de desarrollo de los navegadores.
- Esto debería de ser una herramienta de desarrollo y eliminar estos mensajes en producción.

TU CAM

Mostrar mensajes

Mostrar mensajes en el HTML. Es la forma más compleja y recomendable de mostrar mensajes al usuario.

Para ello necesitamos:

- Asegurar que el código HTML del elemento que queremos utilizar se ha cargado previamente.
- Conocer una manera de acceder al elemento y modificar su contenido. Por ejemplo, con el atributo ID.

```
<div id="mensaje"></div>
<script>
  document.getElementById("mensaje")="Introducir mensaje para el usuario";
</script>
```

TU CAM

PEDIR DATOS AL USUARIO

Pedir datos al usuario

A partir de una ventana emergente:

```
let respuesta = window.prompt("mensaje a mostrar al usuario");
```

La variable respuesta recibirá el valor introducido por el usuario.

- “respuesta” será un texto aunque el usuario introduzca un número
- Si el usuario no escribe nada, “respuesta” será una cadena vacía
- Si el usuario pulsa en cancelar, “respuesta” será igual a “null”
- Esta opción es la menos recomendable. Poco usable y detiene la ejecución hasta que se acepta

TU CAM

Pedir datos al usuario

A través de un input:

Es la forma más amigable de solicitar datos al usuario.

```
<head>
  <script>
    function leerInput(){
      let texto = document.getElementById("entrada").value
      document.getElementById("resultado").innerHTML=texto;
    }
  </script>
</head>
<body>
  <input type="text" id="entrada">
  <button onclick="leerInput();">LEER</button>
  <div id="resultado"></div>
</body>
```

TU CAM

EVENTOS

Eventos

Los eventos de JavaScript se utilizan para detectar acciones que hace el navegador o el usuario.

Los eventos principales son los siguientes:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

https://www.w3schools.com/js/js_events.asp

TU CAM

¡MUCHAS GRACIAS!

TU CAM